

Gujarat IVMS - Integrated Video Management & Analytics

- Hybrid architecture: Model 1 (Registry+GIS) + Model 2 (Unified Viewing) + Model 3 (Federation) + selective Model 4 (Central analytics)
- Live: <https://guj-ivms.vercel.app> | API: <https://guj-ivms-api.onrender.com> | github.com/vaibhav700c/guj-ivms
- 100% free & open-source stack - zero vendor lock-in

1. Problem Understanding + Key Challenges

- 80,000+ CCTV cameras across Gujarat owned by many departments/VMS vendors
- No unified registry, no cross-camera correlation, no common analytics layer
- Centralizing raw video is prohibitively expensive (bandwidth + storage)
- Hackathon scope: prove the platform end-to-end on ~50 cameras

2. Proposed Model - Hybrid (Model 2 + 3 + selective 4)

- Model 1: Registry + GIS - mandatory foundation; every camera catalogued & geolocated
- Model 2: Unified Viewing - RTSP/ONVIF/HLS aggregation for ~50 cameras
- Model 3: Federation Middleware - adapters + event bus; departments keep their VMS
- Model 4 (selective): centralize analytics RESULTS and alerts, not video

3. Why Hybrid - Justification Matrix

- No rip-and-replace: connectors to Sentinel / departmental VMS instead
- ~99% bandwidth reduction: only metadata + snapshots flow to the center
- Fast hackathon demo (direct feeds) that scales to 80K (federation)
- All-open-source keeps software licensing cost at Rs 0

4. High-Level Architecture Diagram

- Camera - Edge/Regional Analytics Node - Metadata + Alerts - Central Platform
- Edge: ANPR, detection, tracking, face recognition (YOLOv8, ByteTrack, OCR)
- Center: FastAPI, PostgreSQL(+PostGIS), Redis, WebSockets, Leaflet GIS
- Streaming: MediaMTX RTSP to WebRTC/HLS; Snapshots: MinIO/S3

5. End-to-End Workflow

- Ingest (edge) to persist (PostgreSQL) to correlate (watchlist) to alert (WS) to act
- Demo simulator emulates edge nodes so the deployed product runs live
- All flows verified end-to-end against the live production deployment

6. Camera Analytics Tiering Strategy

- Tier A (21 cams): ANPR + Face + Detection @ 5-10 FPS
- Tier B (16 cams): Detection + Tracking @ 2-5 FPS
- Tier C (13 cams): Presence / health monitoring @ 1 FPS
- Analytics at the edge keeps compute costs proportional to value

7. ANPR Pipeline - Indian Plates

- YOLOv8 vehicle detection -> plate localization -> OCR with Indian-plate preprocessing
- Normalization: GJ 01 AB 1234 -> GJ01AB1234; OCR-tolerant fuzzy matching
- Detections enrich VAHAN-like registry lookups (owner, maker, RTO)

8. Watchlist Correlation & Alert Generation

- Vehicle & person watchlists (stolen, blacklisted, wanted, missing) with severity + FIR
- Exact & probable (threshold) plate matches; face-name similarity for persons
- Real-time WebSocket push; acknowledge/resolve workflow with sound

9. Vehicle Route Reconstruction Demo

- Search any plate -> timeline of all ANPR sightings across cameras
- Haversine journey legs: distance, elapsed minutes, implied speed
- Leaflet map replay with animated polyline + probable OCR matches

10. GIS Dashboard & Registry

- 50 cameras plotted on OSM tiles across 15 districts
- Status/tier coloring, coverage zones, gap-analysis report
- geo/nearby + geo/coverage endpoints power deployment planning

11. Cross-Camera Tracking (ANPR + ReID)

- Plate-based matching across cameras is primary
- Vehicle ReID (OSNet) prepared for no-ANPR cameras (module scaffolded)
- Journey reconstruction merges sighting + probable-match evidence

12. Scalability Strategy - to 80,000 cameras

- Edge (per-district) -> Regional nodes (MediaMTX + GPU) -> Central DC
- API replicas + Redis pub/sub fan-out (REDIS_URL ready)
- Metadata-only flow keeps bandwidth ~120 KB/min per camera

13. Security Architecture

- JWT (HS256) + RBAC roles (admin/operator/analyst/viewer)
- PBKDF2-SHA256 credential hashing; ingest API key for federation adapters
- Raw video never transits this API - metadata + snapshot refs only

14. Technology Stack - All Free/Open-Source

- FastAPI - SQLAlchemy 2 - PostgreSQL+PostGIS - Redis - SQLite (dev)
- React 18 + Vite + TypeScript + Tailwind CSS - Leaflet - Recharts
- Docker Compose (Postgres, Redis, MinIO, MediaMTX) - Nginx

15. Deployment Architecture (Edge-Regional-Central)

- Render: backend Docker + managed PostgreSQL (free tier)
- Vercel: static frontend, git-connected auto-deploy
- GitHub push to master -> both platforms redeploy automatically

16. Cost-Benefit Analysis - Rs 0 licensing

- Software licensing cost: Rs 0 across the entire stack
- Free tiers on Vercel + Render host the live demo
- Production: existing state DC + GSWAN backbone; GPU ~Rs 150-300/camera/mo

17. Integration VAHAN, SARTHI, eGujCop, AFIS

- VAHAN-like registry simulated with representative dataset (plan 19.2)
- eGujCop/CCTNS-style FIR linkage surfaces in watchlist + alerts
- REST APIs / DB views / batch sync documented for production

18. Key Innovations & Differentiators

- Hybrid model with metadata-only centralization (not raw video)
- Demo simulator makes the deployed product live out-of-the-box
- Journey replay, OCR-fuzzy matching, face-recognition alerts, gap analysis

19. Demo & Next Steps

- Live URLs in README; demo script in docs/DEMO_SCRIPT.md
- Next: connect real Sentinel grid (ingest API + key), ONVIF discovery
- Adopt production DB (Timescale hypertables), MinIO snapshots, Redis fan-out